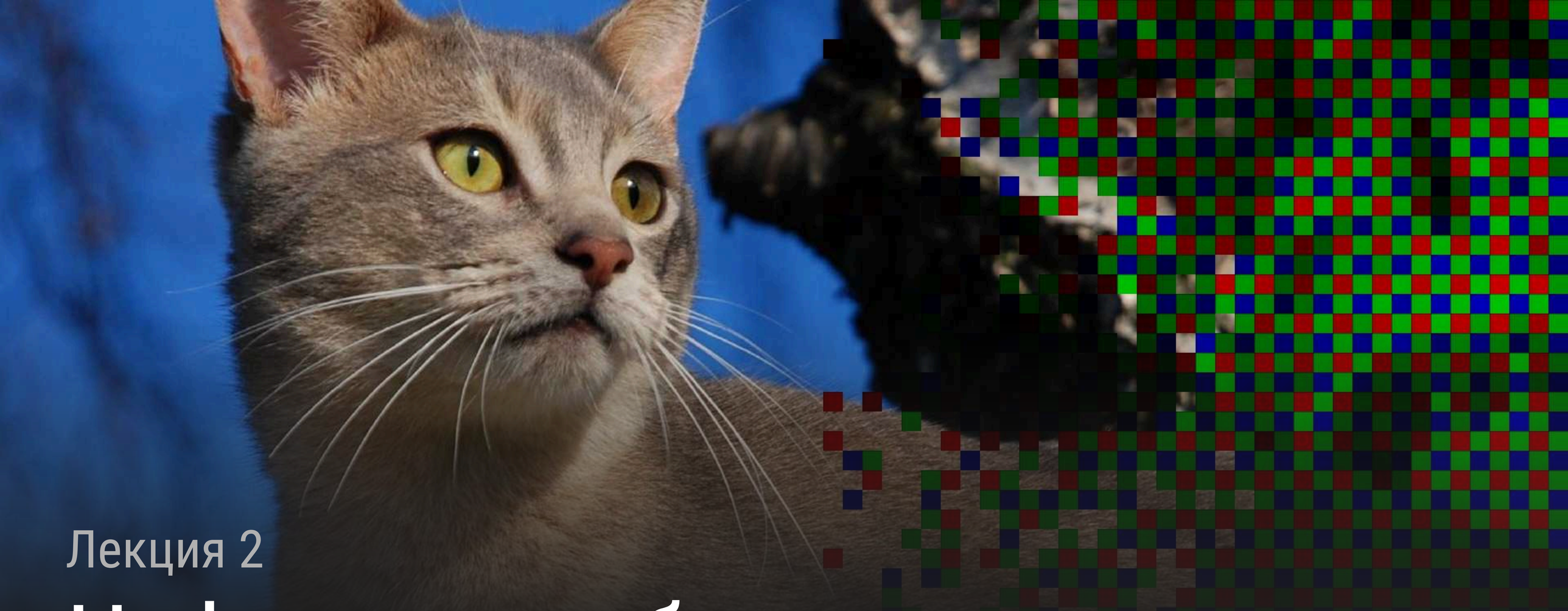


Лекция 2

Цифровое изображение, цвет и фильтрация

План на сегодня

1

Цвет

RGB, HSV, Lab и зачем их менять

2

Гистограммы

Яркость, контраст и выравнивание

3

Шум и фильтры

Свёртка, сглаживание, резкость

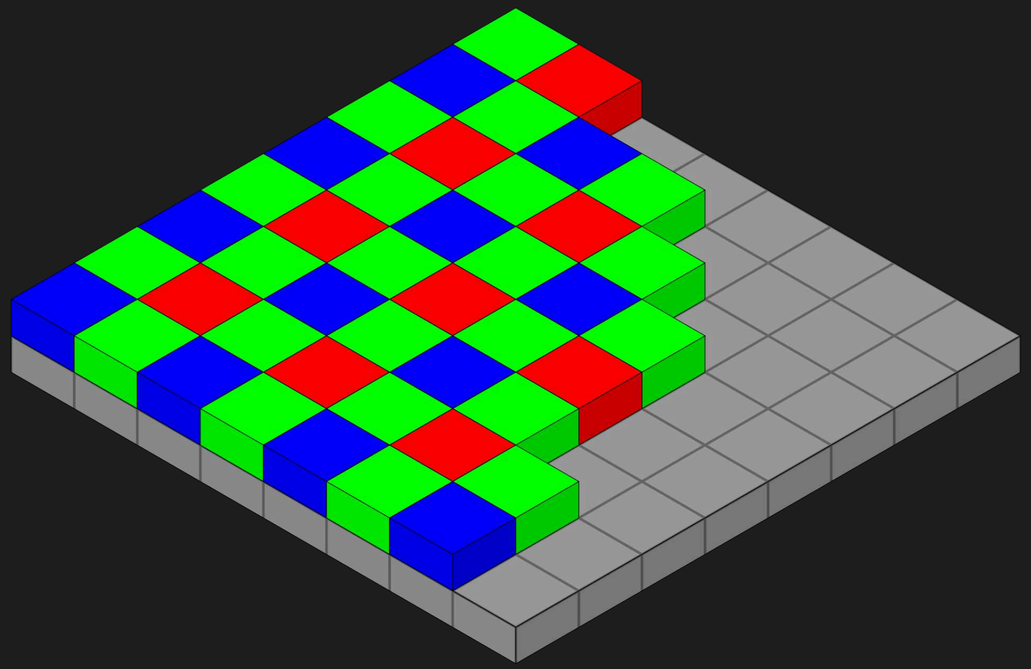
4

Практика

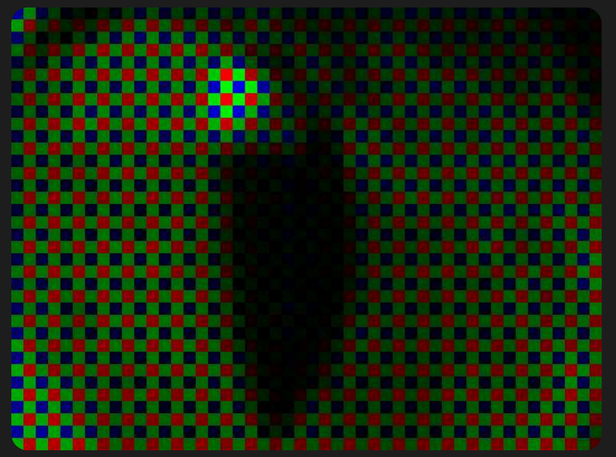
Лаба 2, часть 1: свои фото и свой шум

Откуда берутся числа

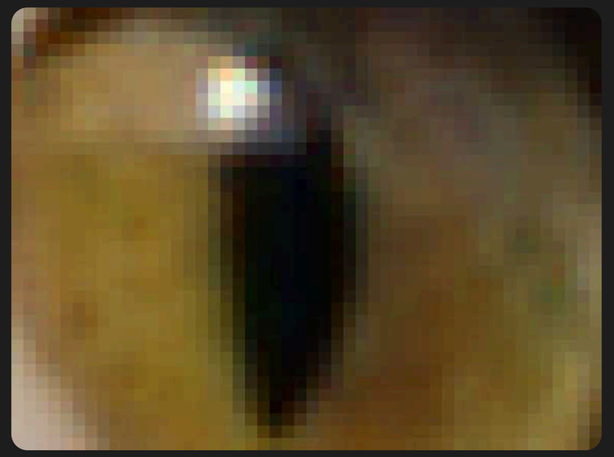
Каждая ячейка матрицы видит один цвет, два других камера достраивает по соседям



Фильтр Байера (Kodak, 1976): половина ячеек зелёные, по четверти красные и синие



что записала матрица



после демозаикинга



фрагмент 48×36 пикселей у глаза кошки из датасета

```
cv2.cvtColor(raw,
cv2.COLOR_BayerBG2RGB)
```

восстановление по мозаике: PSNR 37,2 дБ

Глубина цвета

uint8 хранит 256 уровней на канал, RAW с матрицы 12–14 бит: от 4096 до 16 384 уровней



8 бит: 256 уровней

75 854 разных цвета в кадре



3 бита: 8 уровней

132 цвета, на небе видны полосы



2 бита: 4 уровня

33 цвета

RGB смешивает яркость и цвет

Одна точка на шерсти: обычный кадр и тот же кадр при свете в 5 раз слабее



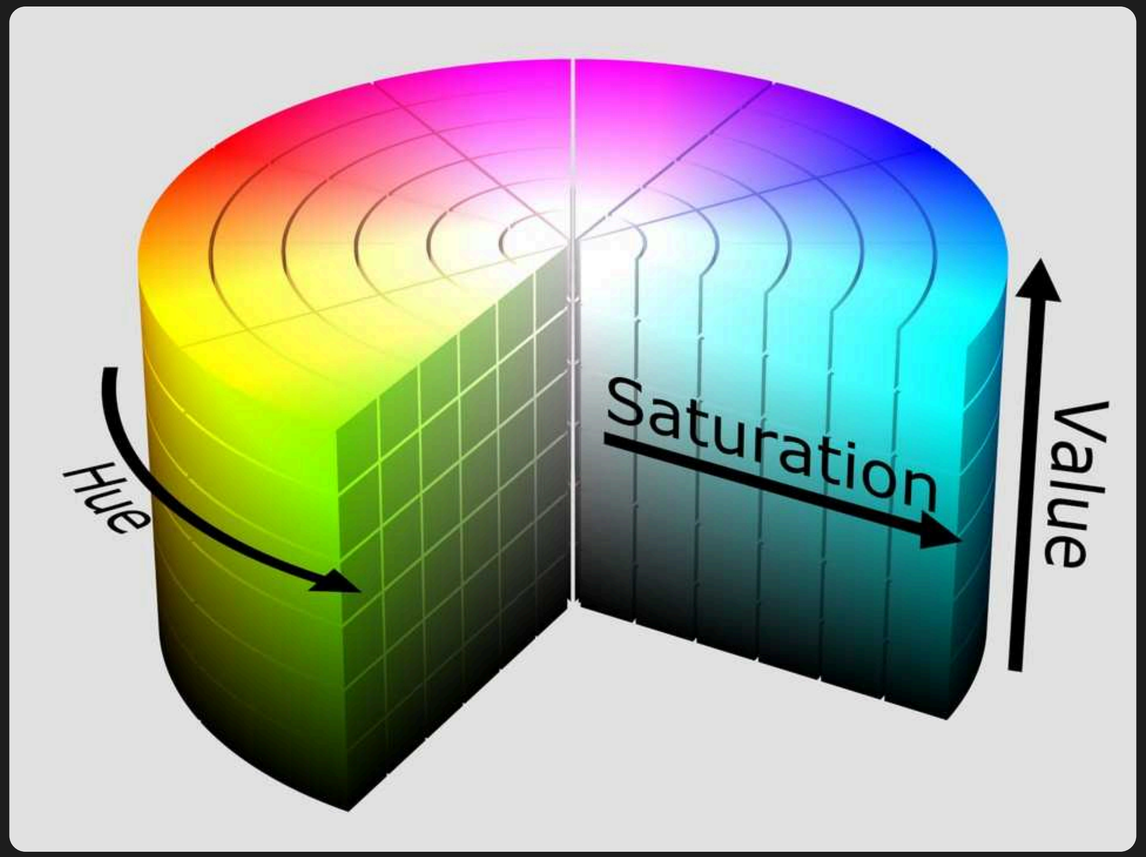


RGB	HSV	Lab
178 130 82	15 138 178	148 141 161
81 59 37	15 139 81	68 135 145

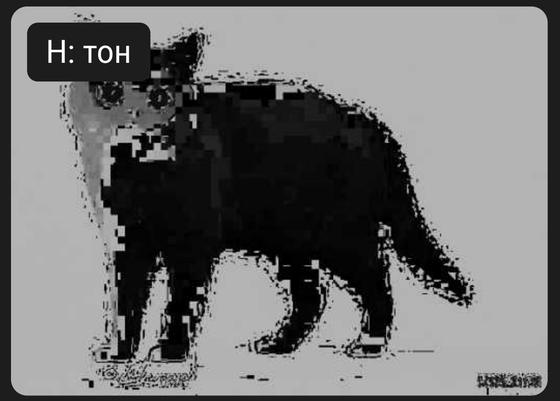
красным числа, которые сдвинулись на 20 и больше

HSV

Тон H, насыщенность S и яркость V лежат в разных каналах











Выделение по цвету

Трава занимает узкий диапазон тона H, маску строит cv2.inRange



исходное фото



маска: 61% пикселей кадра



всё, кроме травы

```
hsv = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)
mask = cv2.inRange(hsv, (30, 60, 40), (90, 255, 255))
```

H в OpenCV идёт от 0 до 179: градусы поделены на 2, чтобы влезть в uint8

Lab

L хранит светлоту, a и b хранят цвет: зелёный–красный и синий–жёлтый



фото



L: светлота



a: зелёный красный



b: синий жёлтый

В канале a собака 124, трава 103: их разделяет одно число

Евклидово расстояние в Lab (ΔE) близко к тому, как разницу цветов видит человек: ΔE около 2 едва заметна глазу

Какое пространство выбрать

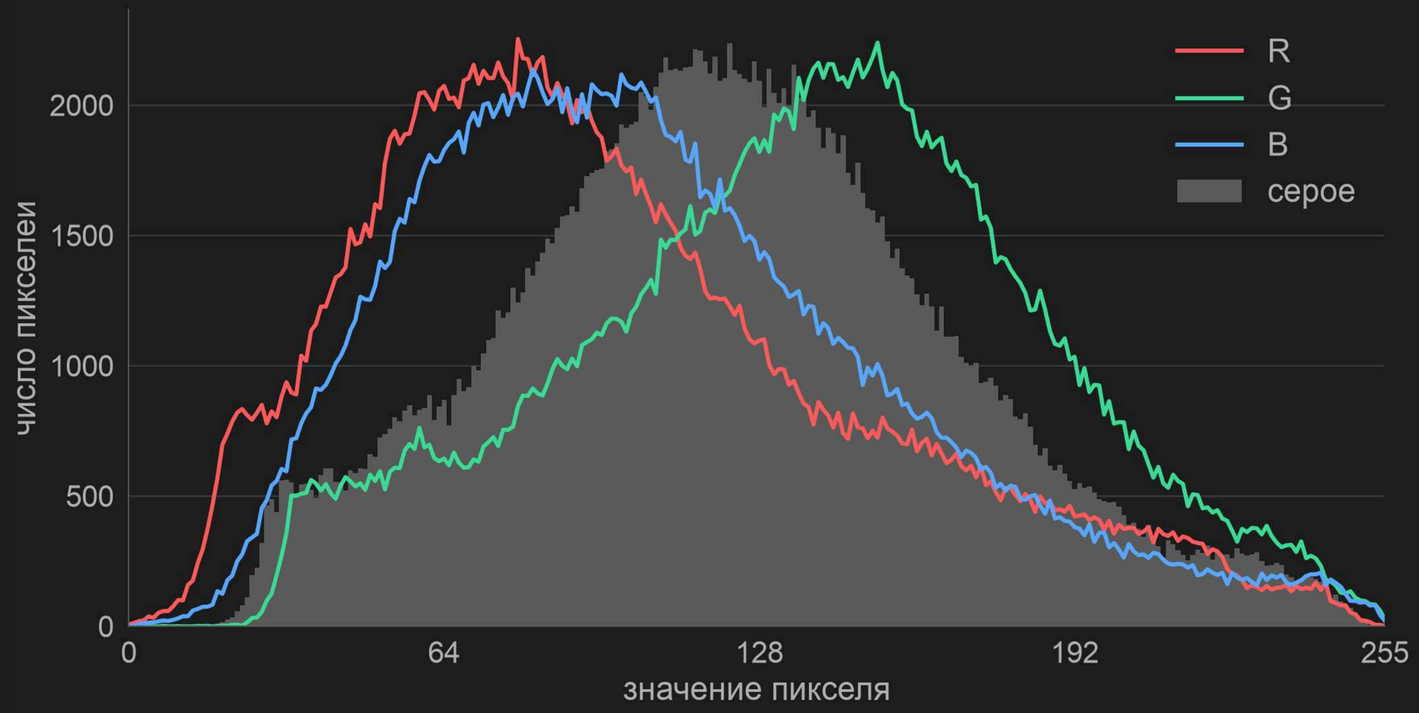
Серое	Один канал яркости: пороги, границы, текстура. Данных в 3 раза меньше	`cv2.COLOR_RGB2GRAY`
HSV	Выделение по цвету через cv2.inRange. Тон H слабо зависит от освещения	`cv2.COLOR_RGB2HSV`
Lab	Коррекция яркости по L без сдвига цвета, сравнение цветов через ΔE	`cv2.COLOR_RGB2LAB`
YCrCb	Y яркость, Cr и Cb цвет: так хранят JPEG и видео, по Cr и Cb ищут кожу	`cv2.COLOR_RGB2YCrCb`

Гистограмма

Сколько пикселей принимает каждое значение от 0 до 255



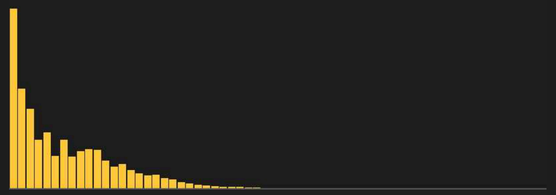
512 × 460 = 235 520 пикселей




```
cv2.calcHist([gray], [0], None, [256], [0, 256])
```

Как читать гистограмму

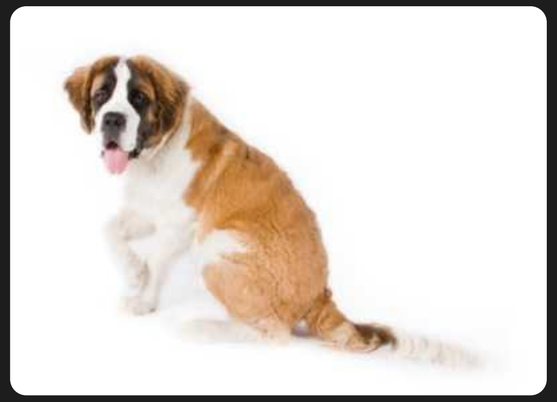


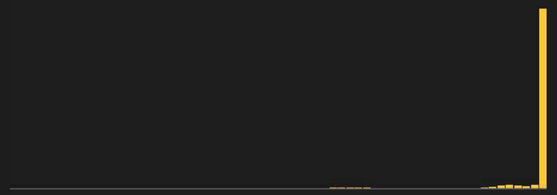


тёмное

горб прижат к нулю

`mean_L 28, std_L 29`



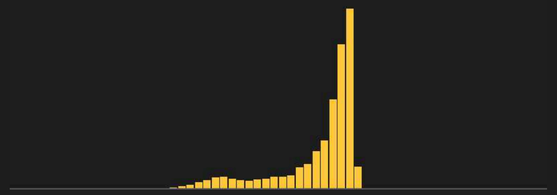


светлое

горб прижат к 255

`mean_L 238, std_L 39`



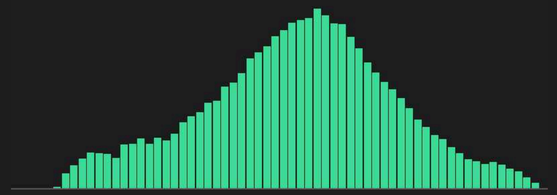


малоконтрастное

узкий горб посередине

`mean_L 146, std_L 22`





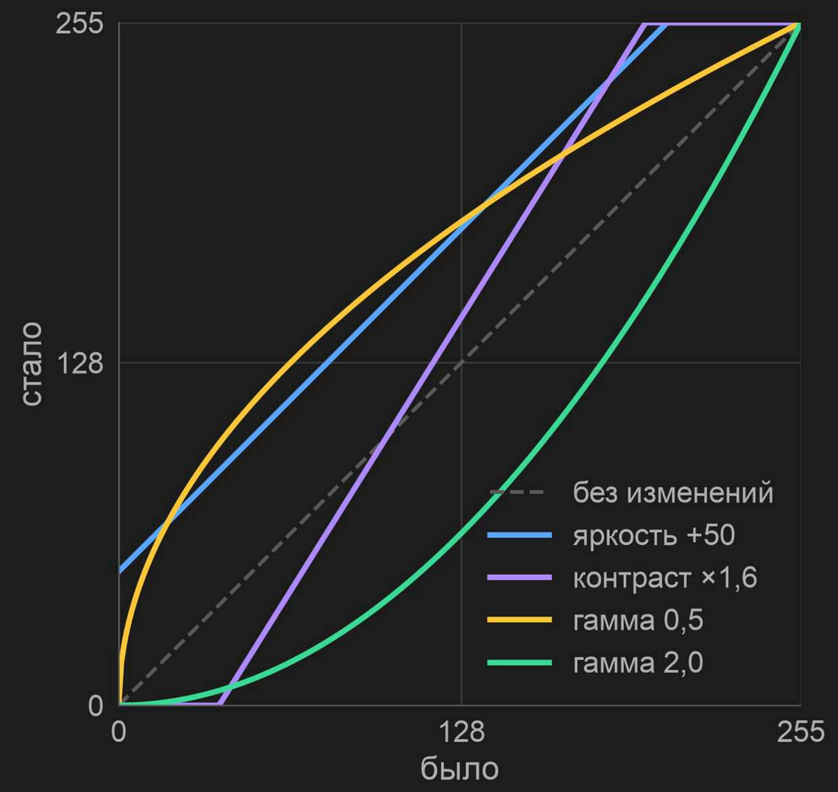
нормальное

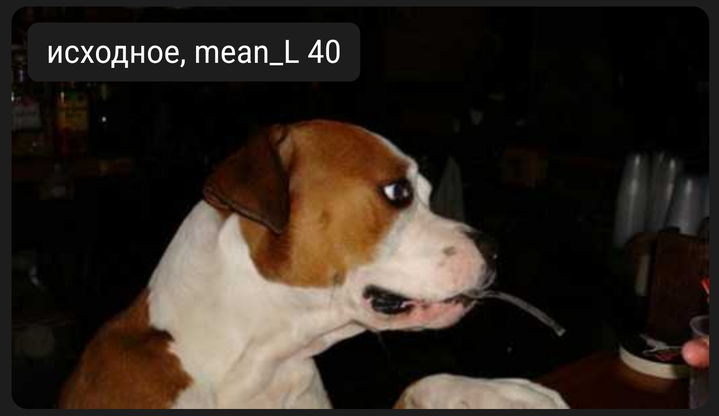
занят почти весь диапазон

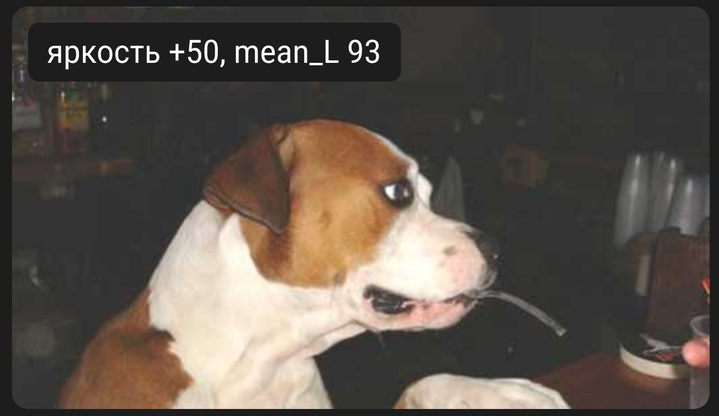
`mean_L 136, std_L 46`

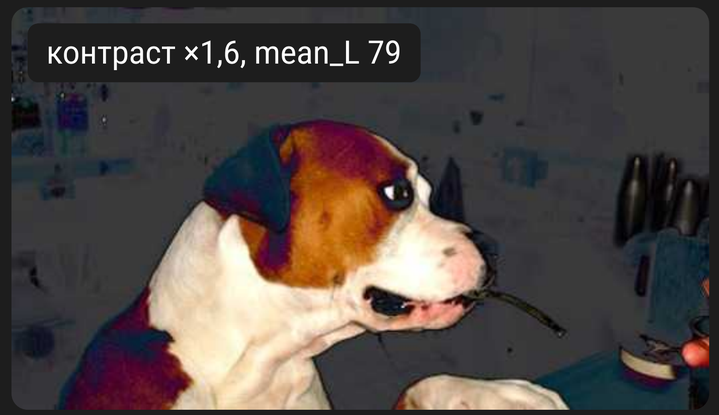
Точечные операции

Новое значение пикселя зависит только от его старого значения











Ловушка uint8

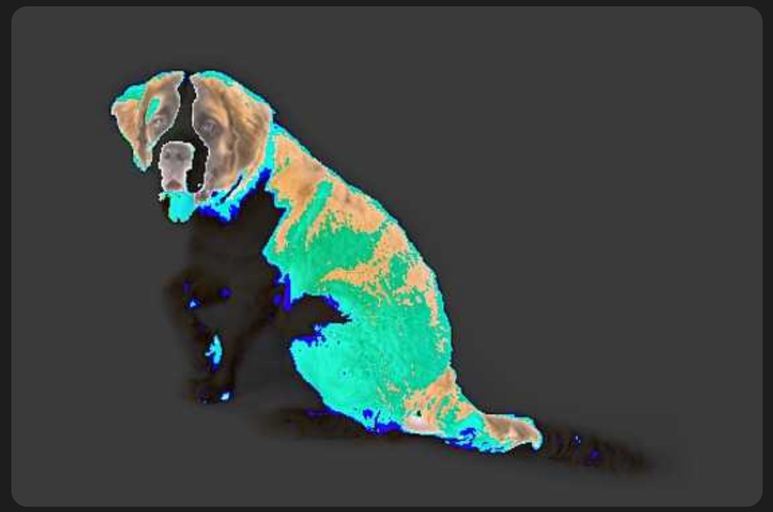
numpy складывает по модулю 256, OpenCV обрезает результат до 255

```
img = np.full((2, 2), 250, np.uint8)
img + 10            # [[4, 4], [4, 4]]
cv2.add(img, 10)    # [[255, 255], [255, 255]]
```

Перед арифметикой переводите массив в float32 или int16, после неё обрезайте через np.clip(x, 0, 255)



исходное



img + 60: переполнение у 93,5% пикселей



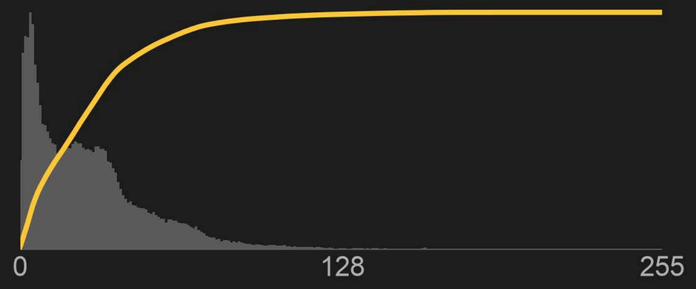
cv2.add(img, 60)

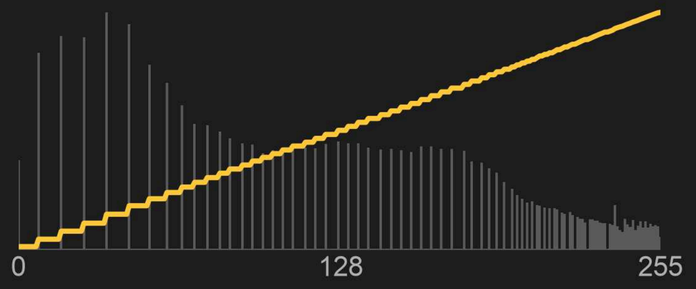
Выравнивание гистограммы

Значения растягиваются так, чтобы накопленная гистограмма (CDF) стала прямой линией









среднее	28	128
разброс	26	74

Жёлтая линия: CDF, доля пикселей со значением не больше данного. После выравнивания она почти прямая

```
eq = cv2.equalizeHist(gray)
```

Выравнивание цветного фото

По каждому каналу RGB цвета плывут, по каналу L в Lab они остаются на месте



исходное

mean_L 28



по R, G, B

mean_L 134, сдвиг цвета 18,5



по L в Lab

mean_L 127, сдвиг цвета 0,6

сдвиг цвета: среднее |Δa| + |Δb| по пикселям относительно исходного кадра

CLAHE

Выравнивание по кусочкам сетки 8×8 с ограничением усиления clipLimit



исходное

mean_L 40



equalizeHist по L

mean_L 138, в тенях вылез шум



CLAHE по L

mean_L 50, фон остался тёмным

```
cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(L)
```

Откуда берётся шум

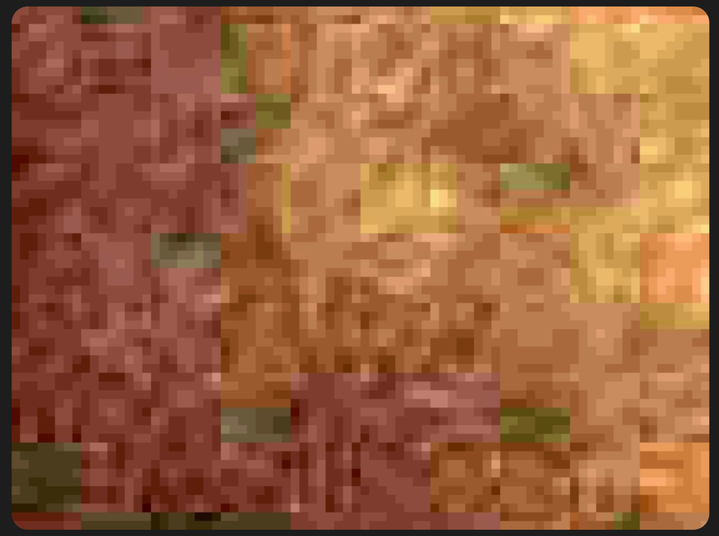
Тёмные места фото из датасета, яркость поднята в 7 и 16 раз



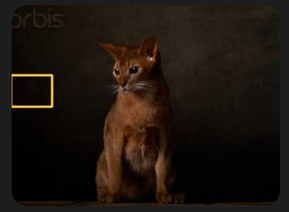
цветной шум в тени



scottish_terrier_171.jpg, яркость ×6,9



блоки JPEG 8×8



Abyssinian_114.jpg, яркость ×15,8

Дробовой шум

фотоны приходят случайно, разброс около √N

Тепловой шум

электроны от нагрева матрицы, растёт с выдержкой

Шум считывания

ошибки усилителя и АЦП при чтении заряда

Усиление ISO

поднимает сигнал вместе со всеми шумами

Сжатие JPEG

блоки 8×8 и ореолы у резких границ

Битые пиксели

ячейка всегда светится или всегда тёмная

Модели шума

Гауссов шум сдвигает каждый пиксель понемногу, импульсный заменяет 5% пикселей на 0 или 255



исходное

эталон



гауссов шум, σ = 20

PSNR 22,2 дБ, SSIM 0,57



импульсный шум, 5%

PSNR 18,4 дБ, SSIM 0,50

Как измерить шум и качество

PSNR считает среднюю ошибку по пикселям, SSIM сравнивает локальную структуру



эталон



шум σ = 16

PSNR 24,1 дБ, SSIM 0,65



яркость +16

PSNR 24,1 дБ, SSIM 0,99

$$PSNR = 10 \times \log10(255^2 / MSE)$$

estimate_sigma: 11,7 на шумном кадре, 3,1 на эталоне

Точечные и локальные операции

Точечной операции нужен один пиксель, локальной нужна его окрестность

точечная: +50

149	118	73	58	62
150	122	74	59	59
147	127	71	58	65
140	131	73	51	64
135	130	85	50	63

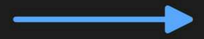


199	168	123	108	112
200	172	124	109	109
197	177	121	108	115
190	181	123	101	114
185	180	135	100	113

Точечные: яркость, контраст, гамма

локальная: среднее 3×3

149	118	73	58	62
150	122	74	59	59
147	127	71	58	65
140	131	73	51	64
135	130	85	50	63

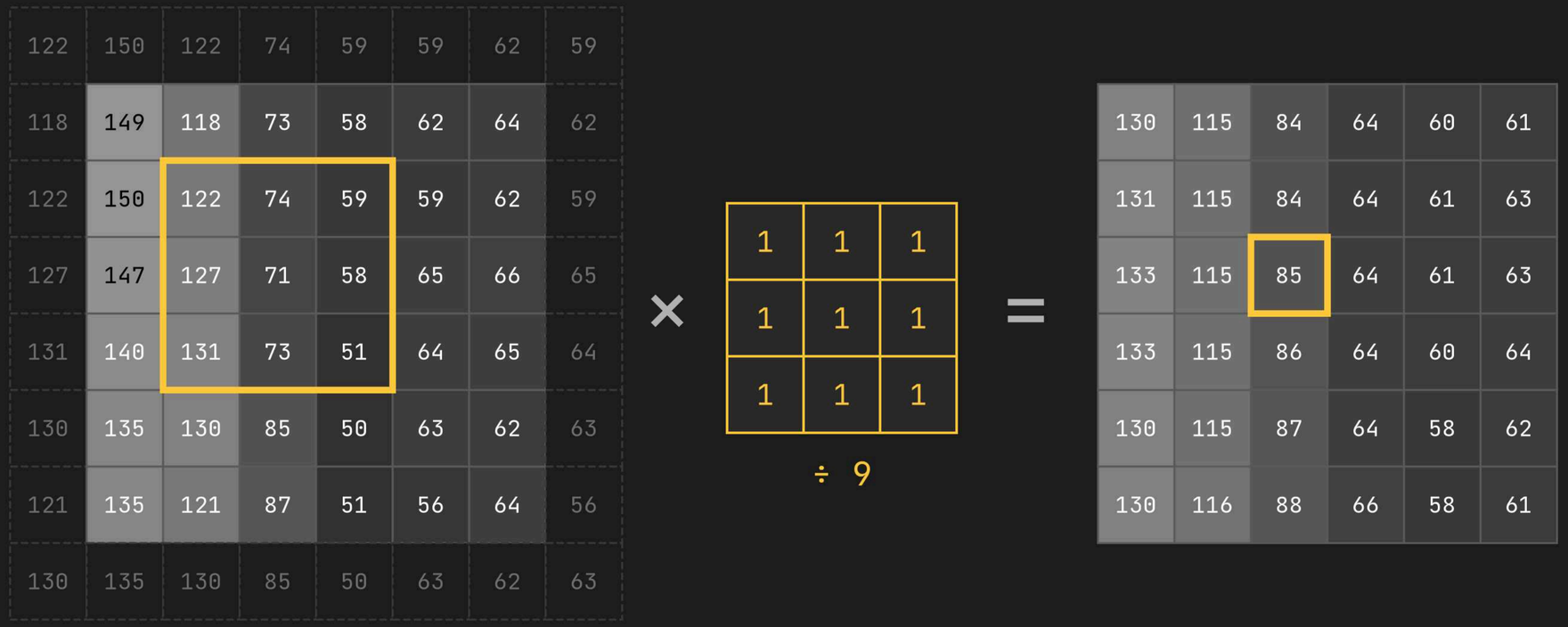


135	113	84	65	64
138	115	84	64	61
138	115	85	64	61
136	115	86	64	60
135	115	87	64	58

Локальные: размытие, резкость, медиана, поиск границ

Свёртка

Ядро скользит по кадру, в каждой позиции считается сумма произведений



Серые клетки по краю: отражённые соседи (BORDER_REFLECT_101). Без них края кадра не посчитать

cv2.filter2D с этим ядром совпадает с cv2.blur(img, (3, 3)) во всех пикселях

```
(122 + 74 + 59 + 127 + 71 + 58 + 131 + 73 + 51) / 9 = 766 / 9 ≈ 85
```

Сглаживающие ядра

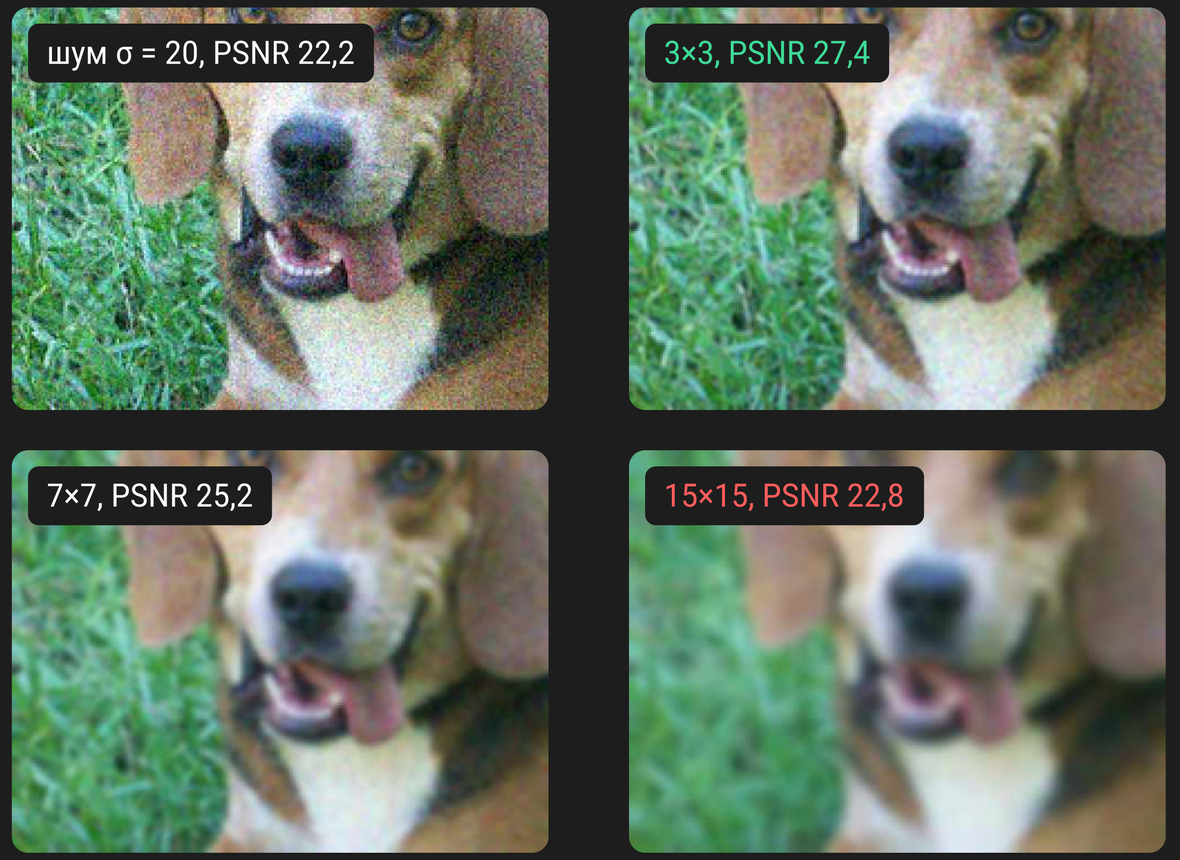
Box усредняет соседей поровну, гауссово ядро даёт центру больший вес

box 5×5

0,04	0,04	0,04	0,04	0,04
0,04	0,04	0,04	0,04	0,04
0,04	0,04	0,04	0,04	0,04
0,04	0,04	0,04	0,04	0,04
0,04	0,04	0,04	0,04	0,04

гаусс 5×5

0,00	0,02	0,02	0,02	0,00
0,02	0,06	0,09	0,06	0,02
0,02	0,09	0,14	0,09	0,02
0,02	0,06	0,09	0,06	0,02
0,00	0,02	0,02	0,02	0,00



Чем больше ядро, тем сильнее размытие: шум уходит вместе с деталями

```
cv2.GaussianBlur(img, (5, 5), 0)
```

Медианный фильтр

Медиана берёт средний по порядку элемент окна, выбросы 0 и 255 в ответ не попадают



импульсный шум 5%

PSNR 18,4 дБ



GaussianBlur 5×5

PSNR 25,2 дБ



medianBlur 3

PSNR 28,5 дБ, SSIM 0,90

```
окно: 112 118 115 255 117 120 255 116 119
```

среднее 147,4, медиана 118

Билатеральный фильтр

Сосед влияет на результат, только если он близко и похож по яркости



шум σ = 20

PSNR 22,2 дБ, SSIM 0,57



GaussianBlur 9×9

PSNR 24,5 дБ, SSIM 0,71



bilateralFilter(9, 75, 75)

PSNR 28,2 дБ, SSIM 0,83

Карло Томази и Роберто Мандучи, 1998: вес соседа = близость по расстоянию × близость по яркости

Сравнение фильтров

30 фото из датасета, 512 px по длинной стороне, среднее по кадрам

	гауссов шум, σ = 20		импульсный шум, 5%		время
	PSNR, дБ	SSIM	PSNR, дБ	SSIM	мс на кадр
`без фильтра`	22,5	0,422	17,9	0,380	—
`blur 5×5`	27,8	0,719	26,3	0,639	0,1
`GaussianBlur 5×5`	29,2	0,764	26,5	0,650	0,1
`medianBlur 3`	27,8	0,673	33,2	0,915	0,1
`medianBlur 5`	27,9	0,702	29,7	0,814	0,2
`bilateralFilter(9, 75, 75)`	30,1	0,788	17,9	0,329	1,4
`fastNlMeansDenoisingColored`	26,9	0,673	18,5	0,388	150,2

Резкость

Unsharp mask: к кадру прибавляют его разницу с размытой копией



мягкий кадр

резкость 78, шум 0,6



unsharp mask

резкость 404, шум 1,2



шум σ = 6, тот же unsharp

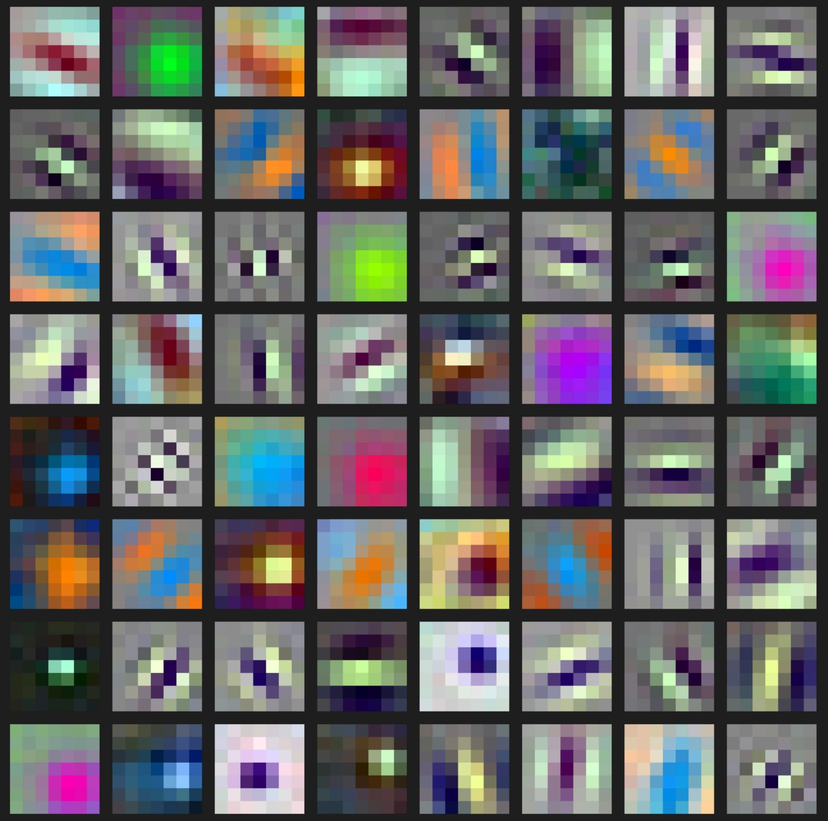
шум 4,0 10,1

```
sharp = cv2.addWeighted(img, 2.5, blur, -1.5, 0)
```

резкость: дисперсия cv2.Laplacian, шум: estimate_sigma, как в лабе

Свёртка в нейросетях

В свёрточной сети ядра не задают руками, их подбирает обучение



ResNet-50, первый слой: 64 ядра 7×7×3, 9408 чисел

Многие ядра похожи на то, что мы собирали руками: перепады яркости в разных направлениях и цветовые пятна

```
nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3)
```

веса conv1 из torchvision, модель обучена на ImageNet

Где это в промышленности

Те же функции OpenCV работают перед нейросетью или вместо неё



рентген, исходный



после CLAHE

Снимок: Stillwaterising, CC0, Wikimedia Commons

```
cv2.createCLAHE(2.0, (8, 8)).apply(xray)
```

Медицина

CLAHE на рентгене и снимках глазного дна

Видеонаблюдение

шумоподавление ночных кадров с высоким ISO и ИК-подсветкой

Сортировка на конвейере

отбраковка ягод, зерна и пластика по цвету в HSV

Документы

выравнивание освещения и шумоподавление перед OCR

Контроль качества

сглаживание перед поиском царапин и трещин на металле

Лаба 2, часть 1

Практика 2: свои фото, свой шум, выбор фильтра

1. **Статистика**
   mean_L, std_L, шум и резкость каждого фото
2. **Гистограммы**
   распределение mean_L, самое тёмное и самое светлое фото
3. **Освещённость**
   5 самых тёмных: по RGB, по L, CLAHE, гамма
4. **Свёртка руками**
   filter2D с ядром из девяток против cv2.blur
5. **Фильтры**
   шум своего варианта, PSNR, SSIM, время, BEST_FILTER

```
lab02/results/
  part1_stats.csv
  histograms.png
  illumination.png
  filters.csv
  filters.png
```

BEST_FILTER из части 1 понадобится в части 2 после лекции 3

Лаба 2 сдаётся целиком, одним ноутбуком, после части 2



Остались вопросы?